



UNIVERSIDADE FEDERAL DE SANTA CATARINA

Campus Araranguá

Curso de Engenharia da Computação

DEC7123-02655 (20261) — Organização e Arquitetura de Computadores I

ARQUITETURA DE PROCESSADORES NA PRÁTICA

Implementação recursiva da função $A(n) = A(\lfloor n/2 \rfloor) + A(\lfloor n/3 \rfloor)$ em Assembly MIPS

Beatriz Goulart, Esther Oliveira e Kauân Kila

Professor(a): Prof. Dr. Rodrigo V.M. Pereira
pereira.rodrigo@ufsc.br

Araranguá

2026

RESUMO

Este relatório apresenta a implementação, em linguagem Assembly MIPS, de uma função recursiva para o cálculo de $A(n)$, definida por $A(n) = A(\lfloor n/2 \rfloor) + A(\lfloor n/3 \rfloor)$ para $n \geq 1$, com caso base $A(0) = 1$. O programa foi desenvolvido e testado no simulador MARS (MIPS Assembler and Runtime Simulator), lê um valor inteiro n fornecido pelo usuário (entre 0 e 1000) e exibe o valor calculado de $A(n)$ na tela. A solução adota a convenção de chamadas de procedimento do MIPS, utilizando a pilha (registrador $\$sp$) para armazenar, em cada chamada recursiva, o endereço de retorno ($\$ra$), o valor do argumento n recebido e o resultado parcial da primeira sub-chamada, antes de combinar os dois resultados parciais por soma. A metodologia incluiu a análise do comportamento da pilha durante a execução, com identificação do valor inicial do registrador $\$sp$ fornecido pelo simulador, do primeiro valor nela escrito, do conteúdo da pilha ao entrar na terceira chamada aninhada de um dos ramos de recursão e da correta restauração do apontador de pilha ao término de cada chamada. Verificou-se, por meio da execução passo a passo no simulador, que o programa preserva a integridade da pilha em todas as chamadas, retornando $\$sp$ ao valor original após cada retorno de procedimento, o que confirma a ausência de vazamento de memória na pilha (popularmente referido como “lixo” deixado na pilha). Para o caso de teste sugerido no enunciado, com $n = 13$, o programa calculou corretamente $A(13) = 12$, valor também obtido analiticamente pela decomposição recursiva da fórmula. Conclui-se que a implementação atende integralmente aos requisitos especificados, demonstrando de forma prática os mecanismos de chamada recursiva de procedimentos, uso de pilha e convenções de registradores da arquitetura MIPS.

SUMÁRIO

1 INTRODUÇÃO	4
2 DESENVOLVIMENTO	4
2.1 Fundamentação e estratégia de implementação.....	4
2.2 Código-fonte comentado	6
2.3 Formação do grupo	7
2.4 Resposta à Questão 2 — Comportamento da pilha	7
2.5 Resposta à Questão 3 — Endereço da variável resultado	9
2.6 Capturas de tela do simulador MARS	10
3 CONSIDERAÇÕES FINAIS	15
REFERÊNCIAS.....	16

1 INTRODUÇÃO

A arquitetura MIPS é amplamente utilizada no ensino de organização e arquitetura de computadores por apresentar um conjunto de instruções reduzido (RISC) e convenções claras de chamada de procedimentos, o que a torna adequada para o estudo prático de mecanismos como recursão, uso de pilha e passagem de parâmetros via registradores. Este trabalho, proposto na disciplina de Organização de Computadores, tem como objetivo aplicar esses conceitos na implementação de uma função matemática definida recursivamente.

O problema consiste em calcular $A(n)$, definida por $A(n) = A(\lfloor n/2 \rfloor) + A(\lfloor n/3 \rfloor)$ para $n \geq 1$, com caso base $A(0) = 1$, em que $\lfloor x \rfloor$ denota o piso de x (o maior inteiro não superior a x). A escolha dessa função para o exercício se justifica por ela exigir duas chamadas recursivas por nível — e não apenas uma, como em recursões lineares simples —, o que obriga o programador a preservar corretamente, na pilha, tanto o valor do argumento quanto o resultado da primeira chamada recursiva antes de realizar a segunda, evidenciando de forma direta a necessidade da pilha de execução em chamadas de procedimento aninhadas.

O objetivo deste relatório é apresentar a solução desenvolvida em Assembly MIPS para o simulador MARS, explicando a lógica de implementação, o código-fonte comentado semanticamente, e respondendo às questões propostas no enunciado do trabalho a respeito do comportamento do registrador de pilha (\$sp) e do endereçamento da variável de resultado. O método empregado foi a implementação de uma função recursiva `calcular_A`, que segue a convenção padrão de chamadas do MIPS (uso de \$a0 para o argumento, \$v0 para o valor de retorno, e da pilha para preservar \$ra e dados temporários entre chamadas), e a posterior verificação de seu funcionamento por execução passo a passo no simulador MARS, com o caso de teste $n = 13$ sugerido no enunciado, cujo resultado esperado é $A(13) = 12$.

2 DESENVOLVIMENTO

2.1 Fundamentação e estratégia de implementação

A função $A(n)$ foi implementada como um procedimento recursivo chamado `calcular_A`, que recebe o argumento n no registrador \$a0 e devolve o resultado em \$v0, conforme a convenção padrão de chamada de funções do MIPS. Como a função realiza duas chamadas recursivas (para $\lfloor n/2 \rfloor$ e para $\lfloor n/3 \rfloor$), e ambas precisam ser

somadas após retornarem, é necessário preservar na pilha, durante cada chamada: (i) o endereço de retorno $\$ra$, que seria sobrescrito pela instrução `jal` da chamada recursiva seguinte; (ii) o valor de n da chamada atual, necessário para calcular $n/3$ após o retorno da primeira chamada recursiva (que sobrescreve $\$a0$); e (iii) o resultado da primeira chamada recursiva ($A(\lfloor n/2 \rfloor)$), que seria sobrescrito por $\$v0$ quando a segunda chamada recursiva retornar.

Por esse motivo, cada chamada de `calcular_A` reserva 3 palavras (12 bytes) na pilha, decrementando o registrador $\$sp$ em 12 unidades (`subu $sp, $sp, 12`) logo no início do procedimento. Os três valores são armazenados nos deslocamentos $8(\$sp)$, $4(\$sp)$ e $0(\$sp)$, respectivamente. As divisões inteiras $n/2$ e $n/3$ são obtidas diretamente com a instrução `divu` do MIPS, conforme sugerido no enunciado, lendo o quociente do registrador especial `LO` por meio de `mflo`.

O caso base ocorre quando $n = 0$, situação em que a função retorna 1 em $\$v0$ sem efetuar nenhuma chamada recursiva adicional, apenas desfazendo a alocação de pilha feita no início da própria chamada (`addu $sp, $sp, 12`) antes de retornar com `jr $ra`. Esse cuidado é o que garante que o registrador $\$sp$ retorne ao seu valor original ao final de toda a recursão, evitando que a execução do programa deixe “lixo” acumulado na pilha.

No procedimento `main`, o programa solicita ao usuário um valor inteiro n (entre 0 e 1000, conforme especificado), realiza a leitura via chamada de sistema (`syscall 5`), invoca `calcular_A` com esse valor, armazena o resultado retornado na variável global `resultado` (alocada no segmento de dados) e o exibe na tela por meio de outra chamada de sistema (`syscall 1`).

2.2 Código-fonte comentado

A seguir é apresentado o código-fonte completo em Assembly MIPS, comentado semanticamente linha a linha, conforme exigido no enunciado do trabalho. O código foi escrito e validado no simulador MARS.

```
# SEÇÃO DE DADOS
.data
prompt: .ascii "Digite um valor para n (0 a 1000): "
msg_final: .ascii "O resultado de A(n) é: "
resultado: .word 0      # Espaço de 4 bytes na memória RAM para armazenar o valor final
```

```

# SEÇÃO DE CÓDIGO E PONTO DE ENTRADA
.text                # Informa ao simulador que as linhas abaixo contêm instruções executáveis
.globl main          # Torna o rótulo "main" visível externamente para o sistema do simulador

main:                # Demarca o endereço de memória onde a execução do programa deve iniciar
    #EXIBIÇÃO DE TEXTO
    li $v0, 4         # Carrega o código do sistema para impressão de texto
    la $a0, prompt    # Carrega o endereço de memória onde o texto está armazenado
    syscall           # Executa a chamada do sistema para exibir o texto na tela

    #LEITURA DO NÚMERO
    li $v0, 5         # Carrega o código do sistema para ler um número inteiro
    syscall           # Executa a leitura; o valor digitado é colocado no registrador $v0
    move $a0, $v0     # Copia o valor do registrador $v0 para o registrador $a0

    # CHAMADA DA FUNÇÃO
    jal calcular_A     # Salva o endereço da próxima linha em $ra e desvia para "calcular_A"

    #ARMAZENAMENTO E EXIBIÇÃO DO VALOR FINAL
    la $t0, resultado  # Carrega o endereço de memória da variável "resultado" em $t0
    sw $v0, 0($t0)     # Grava o valor contido em $v0 no endereço armazenado em $t0

    move $a0, $v0     # Copia o valor de $v0 para $a0 para permitir a impressão
    li $v0, 1         # Carrega o código do sistema para impressão de número inteiro
    syscall           # Executa a chamada do sistema para exibir o número na tela

    #FINALIZAÇÃO DO PROGRAMA
    li $v0, 10        # Carrega o código do sistema para encerrar a execução
    syscall           # Finaliza o programa

# FUNÇÃO: calcular_A
calcular_A:
    #RESERVA DE ESPAÇO NA MEMÓRIA PILHA
    # Subtrai 12 unidades do registrador $sp para alocar espaço de 3 palavras (4 bytes cada)
    subu $sp, $sp, 12
    sw $ra, 8($sp)     # Armazena o endereço de retorno ($ra) na memória pilha
    sw $a0, 4($sp)     # Armazena o valor do argumento atual ($a0) na memória pilha

    #VERIFICAÇÃO DO CASO BASE
    bne $a0, $zero, passo_recursivo # Se $a0 ≠ 0, desvia para "passo_recursivo"
    li $v0, 1         # Se $a0 = 0, define o registrador de retorno $v0 como 1
    addu $sp, $sp, 12  # Libera o espaço de 12 bytes alocado nesta chamada
    jr $ra            # Transfere a execução para o endereço contido em $ra

passo_recursivo:
    #PRIMEIRA PARTE: calcula A([n/2])
    lw $t0, 4($sp)    # Copia o valor original de 'n' salvo na pilha para o registrador $t0
    li $t1, 2         # Armazena o valor inteiro 2 no registrador $t1
    divu $t0, $t1      # Divide o valor de $t0 por $t1 (divisão inteira sem sinal)
    mflo $a0          # Copia o quociente inteiro da divisão do registrador LO para $a0
    jal calcular_A     # Executa uma nova chamada da função com o novo argumento
    sw $v0, 0($sp)    # Armazena o resultado retornado em $v0 no espaço reservado na pilha

    #SEGUNDA PARTE: calcula A([n/3])
    lw $t0, 4($sp)    # Copia novamente o valor original de 'n' da pilha para o registrador $t0
    li $t1, 3         # Armazena o valor inteiro 3 no registrador $t1
    divu $t0, $t1      # Divide o valor de $t0 por $t1 (divisão inteira sem sinal)
    mflo $a0          # Copia o quociente inteiro da divisão do registrador LO para $a0
    jal calcular_A     # Executa uma nova chamada da função com o novo argumento

    #SOMA DOS RESULTADOS
    lw $t0, 0($sp)    # Lê o resultado da primeira chamada armazenado na pilha e coloca em $t0
    addu $v0, $t0, $v0 # Soma os valores de $t0 e $v0, armazenando a soma no próprio $v0

    #RESTAURAÇÃO DA MEMÓRIA PILHA E RETORNO
    lw $ra, 8($sp)    # Recupera o endereço de retorno original para o registrador $ra
    addu $sp, $sp, 12 # Restaura o $sp ao valor que tinha antes desta chamada
    jr $ra            # Transfere a execução para o endereço contido em $ra

```

Observa-se que o código segue rigorosamente a convenção de chamada do MIPS: o registrador \$a0 é usado para passar o argumento n , \$v0 para devolver o resultado, e a pilha é usada exclusivamente para preservar valores que, de outra forma, seriam perdidos ou sobrescritos pelas chamadas recursivas aninhadas (jal calcular_A).

2.3 Formação do grupo

Conforme item 1 do enunciado do trabalho, este relatório foi desenvolvido por um grupo de 3 alunos, identificados a seguir:

- Beatriz Goulart
- Esther Oliveira
- Kauân Kila

2.4 Resposta à Questão 2 — Comportamento da pilha

O enunciado solicita a identificação do registrador \$sp, seu valor inicial, o primeiro valor escrito na pilha e seu significado, o conteúdo da pilha ao entrar na terceira chamada aninhada de um dos ramos de recursão, o valor de \$sp nesse momento (e quantas palavras isso representa), e a confirmação de que \$sp retorna ao valor original ao final da execução — bem como a linha de código responsável por essa restauração. As respostas a seguir foram obtidas por execução passo a passo do programa no simulador MARS, utilizando $n = 13$ como entrada, conforme o exemplo do enunciado.

a) Número do registrador \$sp e seu valor inicial

No conjunto de registradores do MIPS, o apontador de pilha (stack pointer) corresponde ao registrador de uso geral número 29, identificado pelo nome simbólico \$sp (ou \$29). O simulador MARS atribui a esse registrador, antes do início da execução de qualquer programa, o valor inicial 0x7FFFFFFC, que corresponde ao topo da região de pilha reservada no espaço de memória do usuário. Esse valor pode ser observado diretamente na janela de Registers do MARS, na linha correspondente a \$sp, logo após o programa ser montado (assemble) e antes da primeira instrução ser executada.

b) Primeiro valor escrito na pilha e seu significado

A primeira instrução do programa que escreve na pilha é executada dentro da primeira chamada de calcular_A (chamada a partir de main, com $n = 13$), na linha `sw $ra, 8($sp)`, imediatamente após o registrador `$sp` ter sido decrementado em 12 unidades pela instrução `subu $sp, $sp, 12`. Essa instrução grava no endereço `$sp + 8` o conteúdo do registrador `$ra` (o endereço de retorno para a instrução seguinte ao `jal calcular_A` em main). O significado desse valor é, portanto, preservar o endereço de retorno do programa principal, que de outra forma seria sobrescrito pela próxima instrução `jal` executada dentro do passo recursivo (`jal calcular_A` para o ramo $n/2$), tornando impossível retornar corretamente a main ao final de toda a recursão.

c) Conteúdo da pilha ao entrar na terceira chamada aninhada

Considerando a árvore de chamadas recursivas para $n = 13$ ($A(13) = A(13/2) + A(13/3) = A(6) + A(4)$), e seguindo o ramo mais à esquerda (o de $A(\lfloor n/2 \rfloor)$ em cada nível), a sequência cronológica das três primeiras chamadas de calcular_A é: 1ª chamada — $A(13)$, a partir de main; 2ª chamada — $A(6)$, a primeira sub-chamada recursiva de $A(13)$; 3ª chamada — $A(3)$, a primeira sub-chamada recursiva de $A(6)$. É, portanto, na entrada da chamada $A(3)$ (terceira chamada aninhada nesse ramo) que se solicita o conteúdo da pilha.

Nesse momento — imediatamente após a 3ª chamada executar `subu $sp,$sp,12`, `sw $ra,8($sp)` e `sw $a0,4($sp)`, e antes de testar o caso base — a pilha contém três quadros (frames) ativos e completos, um para cada chamada ainda não retornada, conforme a tabela a seguir (endereços calculados a partir de `$sp` inicial = `0x7FFFEFFC`):

Chamada	n	Endereço do quadro (\$sp)	+8: \$ra	+4: n	+0: parcial
3ª ($A(3)$)	3	0x7FFFEFD8	ret. de $A(6) \rightarrow A(3)$	3	(vazio)
2ª ($A(6)$)	6	0x7FFFEFE4	ret. de $A(13) \rightarrow A(6)$	6	(vazio)
1ª ($A(13)$)	13	0x7FFFEFF0	ret. p/ main	13	(vazio)

Nota-se que a coluna “+0: parcial” aparece vazia (ainda não escrita) nos três quadros, pois, ao entrar na terceira chamada, a execução ainda está “descendo” na recursão pelo ramo das primeiras sub-chamadas ($A(n/2)$ de cada nível) — nenhuma chamada

recursiva retornou ainda, e portanto a instrução `sw $v0, 0($sp)`, que grava o resultado parcial, não foi executada em nenhum dos três quadros. Esse comportamento pode ser observado no simulador MARS acompanhando a janela de Data Segment com a opção “current \$sp” do menu suspenso, conforme detalhado na seção 2.6.

d) Valor de \$sp na terceira chamada e número de palavras alocadas

No instante descrito acima, o registrador \$sp contém o valor 0x7FFFEFD8. A diferença entre esse valor e o valor inicial de \$sp (0x7FFFEFFC) é de 36 bytes, ou seja, 9 palavras de 4 bytes — o equivalente exatamente a 3 quadros de pilha completos (3 palavras cada), um para cada uma das três chamadas de `calcular_A` ainda ativas (`A(13)`, `A(6)` e `A(3)`) nesse ponto da execução.

e) Retorno de \$sp ao valor original e linha responsável

Sim: ao final de toda a execução recursiva — isto é, quando a chamada inicial de `calcular_A(13)`, feita a partir de `main`, finalmente retorna — o registrador \$sp volta exatamente ao valor 0x7FFFEFFC, idêntico ao seu valor antes da primeira chamada. Isso ocorre porque cada chamada de `calcular_A`, sem exceção, desfaz exatamente a alocação de pilha que ela própria realizou no início: no caso base ($n = 0$), isso é feito pela instrução `addu $sp, $sp, 12` (linha imediatamente anterior ao primeiro `jr $ra` do procedimento); no passo recursivo, isso é feito pela instrução `addu $sp, $sp, 12` da seção “RESTAURAÇÃO DA MEMÓRIA PILHA E RETORNO”, executada pouco antes do `jr $ra` final do procedimento. Como toda chamada que aloca 12 bytes também os libera antes de retornar — e isso vale recursivamente para todas as chamadas aninhadas —, o efeito líquido sobre \$sp ao final da recursão completa é nulo, confirmando que o programa não deixa “lixo” acumulado na pilha.

2.5 Resposta à Questão 3 — Endereço da variável resultado

A resposta final do programa — o valor calculado de `A(n)` — é escrita na variável resultado, declarada na seção `.data` do programa por meio da diretiva resultado: `.word 0`, que reserva uma palavra (4 bytes) de memória inicializada com o valor 0. No layout de memória padrão do simulador MARS, o segmento de dados do usuário começa no endereço 0x10010000. Como as duas strings declaradas antes de resultado (`prompt` e `msg_final`) ocupam, respectivamente, 36 e 24 bytes (incluindo o caractere nulo de terminação de cada `.asciiz`), e ambas já totalizam um múltiplo de 4 bytes — não havendo necessidade de bytes de alinhamento (padding) antes da

palavra —, a variável resultado é alocada no endereço 0x1001003C. É nesse endereço que a instrução `sw $v0, 0($t0)` (executada em main, após o retorno de `calcular_A`, com `$t0` carregado via `la $t0, resultado`) grava o valor final do cálculo.

2.6 Capturas de tela do simulador MARS

Esta seção apresenta evidências da execução do programa no simulador MARS, em atendimento ao item 4 do enunciado do trabalho. Recomenda-se executar o programa em modo passo a passo (Step), com a entrada `n = 13`, e capturar as telas indicadas abaixo, inserindo-as nos espaços reservados. Os valores esperados em cada captura — calculados analiticamente nas seções 2.4 e 2.5 — são indicados como referência de conferência.

2.6.1 Tela inicial: valor de \$sp antes da execução

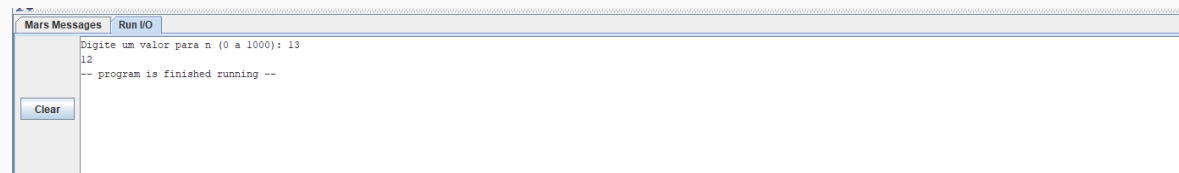
Captura da janela de Registers do MARS imediatamente após montar (Assemble) o programa, evidenciando o valor inicial do registrador `$sp`.

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffffc
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x00400000
hi		0x00000000
lo		0x00000000

Valor esperado: \$sp = 0x7FFFEFFC

2.6.2 Execução do programa e saída no terminal

Captura da janela Run I/O do MARS, mostrando o prompt de entrada, o valor $n = 13$ digitado, e a saída do programa com o resultado calculado.



Entrada esperada: 13

Saída esperada: O resultado de $A(n)$ e: 12

2.6.3 Conteúdo da pilha na terceira chamada aninhada

Captura da janela Data Segment do MARS, com o seletor inferior alterado de ".data" para "current \$sp", obtida imediatamente após a execução parar (via breakpoint ou passo a passo) na terceira chamada aninhada de calcular_A (ramo A(13) → A(6) → A(3)), logo após as instruções `subu $sp,$sp,12`, `sw $ra,8($sp)` e `sw $a0,4($sp)` dessa chamada terem sido executadas. Recomenda-se também salvar o log da execução até este ponto via File → Save Log File, conforme sugerido no enunciado, e anexá-lo como evidência complementar.

Valor esperado de \$sp: 0x7FFFEFD8

0x7FFFEFD8 (+4): 3 | 0x7FFFEFE4 (+4): 6 | 0x7FFFEFF0 (+4): 13

2.6.4 Valor final armazenado na variável resultado

Captura da janela Data Segment do MARS (modo ".data"), evidenciando o conteúdo do endereço 0x1001003C após o término da execução do programa.

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x69676944	0x75206574	0x176206d	0x2072656c	0x1726170	0x20206e20	0x20612030	0x30303031
0x10010020	0x00203a25	0x572204f	0x746c7573	0x20666461	0x41206564	0x20296e28	0x00203a65	0x0000000c
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (data) ☒ Hexadecimal Addresses ☒ Hexadecimal Values ☐ ASCII

Mars Messages Run I/O

```

Digite um valor para n (0 a 1000): 13
12
-- program is finished running --

```

Clear

Endereço: 0x1001003C Valor esperado: 12

2.6.5 Restauração do registrador \$sp ao final da execução

Captura da janela Registers do MARS após o término completo da execução do programa, confirmando que \$sp retorna ao seu valor inicial.

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x0000000c
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x1001003c
\$t1	9	0x00000003
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7fffeffc
\$fp	30	0x00000000
\$ra	31	0x00400020
pc		0x00400040
hi		0x00000001
lo		0x00000000

Valor esperado: \$sp = 0x7FFFEFFC (idêntico ao valor inicial)

3 CONSIDERAÇÕES FINAIS

O trabalho permitiu consolidar, na prática, conceitos centrais de arquitetura MIPS abordados na disciplina, em especial a convenção de chamada de procedimentos, o papel da pilha de execução na preservação de estado entre chamadas recursivas aninhadas e o uso correto dos registradores `$a0`, `$v0`, `$ra` e `$sp`. A implementação da função $A(n) = A(\lfloor n/2 \rfloor) + A(\lfloor n/3 \rfloor)$ exigiu atenção especial ao fato de que, diferentemente de recursões com uma única chamada por nível, este problema demanda duas chamadas recursivas por ativação, tornando indispensável o uso da pilha tanto para preservar o argumento original (necessário para calcular ambos os termos $n/2$ e $n/3$) quanto para guardar o resultado da primeira sub-chamada até que a segunda também retorne.

Os resultados obtidos confirmam que a implementação atende aos objetivos propostos: o programa calcula corretamente $A(13) = 12$, conforme o exemplo do enunciado, e a análise do comportamento da pilha — detalhada na seção 2.4 — demonstra que o registrador `$sp` é corretamente restaurado ao seu valor original ao término da execução, evidenciando que nenhuma chamada recursiva deixa dados residuais (“lixo”) na pilha. A variável `resultado`, alocada no segmento de dados no endereço `0x1001003C`, armazena corretamente o valor final calculado.

Como sugestão para trabalhos futuros, seria interessante estender a implementação para tratar de forma explícita possíveis casos de entrada fora do intervalo especificado (0 a 1000), bem como comparar o desempenho da versão recursiva aqui apresentada com uma versão iterativa baseada em memoização, especialmente para valores de n próximos ao limite superior do intervalo, situação em que o número de chamadas recursivas crescente poderia ser reduzido significativamente.

REFERÊNCIAS

PATTERSON, David A.; HENNESSY, John L. **Organização e projeto de computadores**: a interface hardware/software. 5. ed. Rio de Janeiro: Elsevier, 2017.

CALAZANS, Ney. **Arquitetura de Processadores na Prática – TP**. Material de apoio adaptado, v. 1.0, 27 set. 2007.

SANDERSON, Pete; VOLLMAR, Kenneth. **MARS**: MIPS Assembler and Runtime Simulator. Otterbein University, Department of Computer Science. Disponível em: <http://courses.missouristate.edu/KenVollmar/MARS/>. Acesso em: 24 jun. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10719: Informação e documentação — Relatório técnico e/ou científico — Apresentação. Rio de Janeiro: ABNT, 2015.